



DATAWEAVE FUNCTION

map()

Think Smart!

map() transforms each element of an Array and returns a new Array.

Transforms each element of an Array by applying a logic or expression and returns a new Array.

1. WHAT DOES IT DO?

The `map()` function is used to transform each element of an Array by applying a logic or expression and returns a new Array with the updated values.

2. WHEN DO WE USE IT?

- When we need to transform each item in an Array.
- When we want to change or calculate values for every element.
- When we want to reshape data.

3. SYNTAX

```
payload map ((item, index) -> {
  // transformation logic
})
```

- `item` -> current item of the Array
- `index` -> current index (optional)
- > transformation logic

4. IMPORTANT POINTS

- Always returns a new Array.
- Does not modify the original Array.
- Works on every element individually.
- You can use `item`, `index` or both.

5. EASY EXAMPLE

Input (Payload)

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 28 }
]
```

DataWeave Code

```
output application/json
---
payload map (item -> item.name)
```

Output

```
["John", "Alice", "Bob"]
```

6. MEDIUM EXAMPLE

Input (Payload)

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 28 }
]
```

DataWeave Code

```
output application/json
---
payload map (item -> {
  name: item.name,
  age: item.age + 1
}))
```

Output

```
[
  { "name": "John", "age": 26 },
  { "name": "Alice", "age": 31 },
  { "name": "Bob", "age": 29 }
]
```

7. DIFFICULT EXAMPLE (REAL-TIME SCENARIO)

Requirement:
From an API response, return only active customers with their names in uppercase.

Input (Payload)

```
[
  { "id": 1, "name": "john", "status": "Active" },
  { "id": 2, "name": "alice", "status": "Inactive" },
  { "id": 3, "name": "bob", "status": "Active" }
]
```

DataWeave Code

```
output application/json
---
payload
  filter (item -> item.status == "Active")
  map (item -> {
    id: item.id,
    name: upper(item.name)
  })
```

Output

```
[
  { "id": 1, "name": "JOHN" },
  { "id": 3, "name": "BOB" }
]
```

8. COMMON MISTAKE

Using `map()` on an Object instead of an Array.

```
payload map (item -> item.name)
// Error if payload is an Object
```

Tip: Always make sure your payload is an Array before using `map()`.

9. INTERVIEW TIP

Be ready to explain the difference between `map()` and `mapObject()`.

map()	mapObject()
Works on Arrays	Works on Objects
Example: <pre>[1, 2, 3] map ((item) -> item * 2) // [2, 4, 6]</pre>	Example: <pre>{a:1, b:2} mapObject ((value, key) -> (key): value * 2) // {a:2, b:4}</pre>

10. PRO TIP

- Use `map()` for clean and readable transformations.
- Combine `map()` with `filter()`, `reduce()`, `groupBy()` for powerful transformations.
- Keep your transformation logic simple and reusable.
- Always test with real payloads!

Remember: `map()` is your best friend when you need to transform each element of an Array. Master it, and you can transform any data like a PRO!

DATAWEAVE FUNCTIONS CHEAT SHEET

1. map()

What does it do?

Transforms each element of an Array using a given expression and returns a new Array.

When do we use it?

When we need to modify each element in an Array.

Real-Time Scenario

Convert all customer names to uppercase.

```
payload map (item -> upper(item.name))
```

Syntax

```
payload map ((item, index) -> {
  // transformation logic
})
```

Example

Input (Payload)

```
[ { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 } ]
```

Output

```
[ "JOHN", "ALICE" ]
```

2. filter()

What does it do?

Filters elements of an Array based on a condition and returns a new Array.

When do we use it?

When we want to select only required items from an Array.

Real-Time Scenario

Get only customers whose age is greater than 18.

```
payload filter (item -> item.age > 18)
```

Syntax

```
payload filter (item ->
  // condition
)
```

Example

Input (Payload)

```
[ { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 17 } ]
```

Output

```
[ { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 } ]
```

3. reduce()

What does it do?

Reduces an Array into a single value by applying an accumulator function.

When do we use it?

When we need to calculate totals, sums, counts or merge data.

Real-Time Scenario

Calculate total order amount.

```
payload reduce ((acc, item) -> acc + item.amount, 0)
```

Syntax

```
payload reduce ((acc, item, index) -> {
  // logic
}, initialValue)
```

Example

Input (Payload)

```
[ 10, 20, 30, 40 ]
```

Output

```
100
```

4. distinctBy()

What does it do?

Returns unique elements from an Array based on a key or expression.

When do we use it?

When we want to remove duplicates from an Array based on a field.

Real-Time Scenario

Remove duplicate customers based on id.

```
payload distinctBy $.id
```

Syntax

```
payload distinctBy ((item, index) ->
  item.key)
// or distinctBy $.key
```

Example

Input (Payload)

```
[ { "id": 1, "name": "John" },
  { "id": 1, "name": "John" },
  { "id": 2, "name": "Alice" } ]
```

Output

```
[ { "id": 1, "name": "John" },
  { "id": 2, "name": "Alice" } ]
```

5. orderBy()

What does it do?

Sorts the elements of an Array in ascending or descending order.

When do we use it?

When we need to sort data based on one or more fields.

Real-Time Scenario

Sort customers by age in ascending order.

```
payload orderBy ((item) -> item.age)
```

Syntax

```
payload orderBy ((item) -> item.key)
// or
payload orderBy ((item) -> item.key) desc
```

Example

Input (Payload)

```
[ { "name": "John", "age": 30 },
  { "name": "Alice", "age": 25 } ]
```

Output

```
[ { "name": "Alice", "age": 25 },
  { "name": "John", "age": 30 } ]
```

6. groupBy()

What does it do?

Groups the elements of an Array based on a key and returns an Object.

When do we use it?

When we want to group records by a field.

Real-Time Scenario

Group employees by department.

```
payload groupBy $.dept
```

Syntax

```
payload groupBy ((item, index) ->
  item.key)
// or groupBy $.key
```

Example

Input (Payload)

```
[ { "dept": "HR", "name": "John" },
  { "dept": "IT", "name": "Alice" },
  { "dept": "HR", "name": "Bob" } ]
```

Output

```
{
  "HR": [ { "dept": "HR", "name": "John" },
    { "dept": "HR", "name": "Bob" } ],
  "IT": [ { "dept": "IT", "name": "Alice" } ]
}
```

7. mapObject()

What does it do?

Transforms each value of an Object using a given expression and returns a new Object.

When do we use it?

When we want to modify all values of an Object.

Real-Time Scenario

Multiply all values in the object by 10.

```
payload mapObject ((value, key) -> value * 10)
```

Syntax

```
payload mapObject ((value, key, index) ->
  // logic
)
```

Example

Input (Payload)

```
{ "a": 1, "b": 2, "c": 3 }
```

Output

```
{ "a": 10, "b": 20, "c": 30 }
```

8. isEmpty()

What does it do?

Checks if an Array, Object, String or Null is empty.

When do we use it?

When we need to validate empty payloads or fields.

Real-Time Scenario

Check if the payload is empty before processing.

```
if (isEmpty(payload)) "No Data" else "Data Available"
```

Syntax

```
isEmpty(payload)
// or isEmpty(payload.field)
```

Example

Input (Payload)

```
[ ]
{ }
""
null
```

Output

```
true
```

9. sizeof()

What does it do?

Returns the number of elements in an Array or Object.

When do we use it?

When we need to count items in Array or keys in Object.

Real-Time Scenario

Get the total number of records in the payload.

```
sizeof(payload)
```

Syntax

```
sizeof(payload)
// or sizeof(payload.field)
```

Example

Input (Payload)

```
[ "a", "b", "c", "d" ]
```

Output

```
4
```

10. update

What does it do?

Updates or modifies specific values in an Object or Array without changing the original structure.

When do we use it?

When we want to update specific fields or indices.

Real-Time Scenario

Increase age by 1.

```
payload update { case $.age -> $.age + 1 }
```

Syntax

```
payload update {
  case $.field -> newValue,
  case ${index} -> newValue
}
```

Example

Input (Payload)

```
[ { "name": "John", "age": 25, "city": "NY" } ]
```

Output

```
[ { "name": "John", "age": 26,
  "city": "NY" } ]
```

Pro Tips

- ✓ Always understand the payload structure first.
- ✓ Use variables for readability.
- ✓ Combine functions for powerful transformations.
- ✓ Keep your code simple and maintainable.

Remember!

Understand the data.
Choose the right function.
Transform with confidence!

Common Combo!

filter() -> map() -> orderBy() -> map()
filter() -> groupBy() -> mapObject()
filter() -> reduce()
trim() -> replace() -> default



DATAWEAVE FUNCTION

filter()



Filters elements of an **Array** based on a condition and returns a new **Array**.

Think Smart!



Use `filter()` when you want only the required items from an **Array**.

1. WHAT DOES IT DO?

The `filter()` function is used to filter elements of an **Array** based on a condition or expression and returns a new **Array** containing only the elements that match the condition.



2. WHEN DO WE USE IT?

- ✓ When we need to select only specific items from an **Array**.
- ✓ When we want to apply a condition on one or more fields.
- ✓ When we want to remove unwanted items from data.



3. SYNTAX

```
payload filter (item -> condition)
// or
payload filter ((item, index) -> condition)
```

- **item** -> current item of the **Array**
- **index** -> current index (optional)
- **condition** -> expression that returns **true** or **false**

4. IMPORTANT POINTS

- Always returns a new **Array**.
- Does not modify the original **Array**.
- Condition must return **true** or **false**.
- Works on every element individually.



5. EASY EXAMPLE

Input (Payload)

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 17 },
  { "name": "Mike", "age": 22 }
]
```

DataWeave Code

```
payload filter (item -> item.age >= 18)
```

Output

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Mike", "age": 22 }
]
```



6. MEDIUM EXAMPLE



Input (Payload)

```
[
  { "name": "John", "age": 25, "status": "Active" },
  { "name": "Alice", "age": 30, "status": "Inactive" },
  { "name": "Bob", "age": 17, "status": "Active" },
  { "name": "Mike", "age": 22, "status": "Active" }
]
```

DataWeave Code

```
payload filter (item -> item.status == "Active" and item.age >= 18)
```

Output

```
[
  { "name": "John", "age": 25, "status": "Active" },
  { "name": "Mike", "age": 22, "status": "Active" }
]
```



7. DIFFICULT (REAL-TIME SCENARIO)

Requirement:

From an API response, return only active customers who have placed orders greater than 1000.

Input (Payload)

```
[
  { "id": 1, "name": "John", "status": "Active", "orderAmount": 1200 },
  { "id": 2, "name": "Alice", "status": "Inactive", "orderAmount": 1500 },
  { "id": 3, "name": "Bob", "status": "Active", "orderAmount": 900 },
  { "id": 4, "name": "Mike", "status": "Active", "orderAmount": 2000 }
]
```

DataWeave Code

```
payload filter (item -> item.status == "Active"
  and item.orderAmount > 1000)
```

Output

```
[
  { "id": 1, "name": "John", "status": "Active", "orderAmount": 1200 },
  { "id": 4, "name": "Mike", "status": "Active", "orderAmount": 2000 }
]
```



8. COMMON MISTAKE

Using `filter()` on an **Object** instead of an **Array**.

```
payload filter (item -> item.name == "John")
```

✗ Error if payload is an **Object**



Tip: `filter()` works only on **Arrays**.
Use `mapObject()` for **Objects**.



9. INTERVIEW TIP

Be ready to explain how `filter()` works with different conditions.

- ✓ Single condition
- ✓ Multiple conditions
- ✓ Using logical operators (**and**, **or**, **not**)
- ✓ Filtering based on nested fields
- ✓ Difference between `filter()` and `where()`



10. PRO TIP

- ✓ Use `filter()` early in the transformation to reduce data size.
- ✓ Combine `filter()` with `map()`, `groupBy()`, `orderBy()` for powerful transformations.
- ✓ Keep conditions simple, readable and maintainable.



Remember: `filter()` is your best friend when you need only the **right data**.
Understand the condition. Filter smart. Transform better!





DATAWEAVE FUNCTION

pluck



Extracts the value of a specific **key** from each **Object** in an **Array** and returns a new **Array**.

Think Smart!

Use `pluck()` when you want to quickly extract one field from every Object in an Array.

1. WHAT DOES IT DO?

The `pluck()` function is used to extract the value of a given **key** from each **Object** in an **Array** and returns those values in a new **Array**.



2. WHEN DO WE USE IT?

- ✓ When we want to extract one specific field from every Object in an Array.
- ✓ When we want a quick and clean way to get a list of values.
- ✓ When we need to reduce complex data into a simple Array.



3. SYNTAX

```
payload pluck (key)
// or
payload pluck ((item, index) -> key)
```

- **key** -> The key whose value you want to extract
- **item** -> Current item of the Array
- **index** -> Current index (optional)

4. IMPORTANT POINTS

- Always returns a new Array.
- Does not modify the original Array.
- If the key does not exist, it returns **null** for that item.
- Works only on Arrays of Objects.



5. EASY EXAMPLE

Input (Payload)

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 28 }
]
```

Output

```
["John", "Alice", "Bob"]
```



6. MEDIUM EXAMPLE



Input (Payload)

```
[
  { "id": 1, "name": "John", "age": 25 },
  { "id": 2, "name": "Alice", "age": 30 },
  { "id": 3, "name": "Bob", "age": 28 }
]
```

DataWeave Code

```
payload pluck ("age")
```

Output

```
[25, 30, 28]
```



7. DIFFICULT (REAL-TIME SCENARIO)



Requirement:

From an API response, return all product names from the list of products.

Input (Payload)

```
[
  { "id": 101, "name": "Laptop", "price": 55000 },
  { "id": 102, "name": "Mobile", "price": 25000 },
  { "id": 103, "name": "Headphone", "price": 1500 }
]
```

DataWeave Code

```
payload pluck ("name")
```

Output

```
["Laptop", "Mobile", "Headphone"]
```



8. COMMON MISTAKE

Using `pluck()` on an Object instead of an Array.

Wrong Example

```
payload pluck ("name")
// Error if payload is an Object
```

Tip: Always make sure your payload is an Array of Objects.



9. INTERVIEW TIP

Be ready to explain how `pluck()` works.

Key Points to Mention:

- ✓ It extracts values of a specific key from each Object in an Array.
- ✓ It returns a new Array.
- ✓ It is very helpful for simplifying complex transformations.
- ✓ It is similar to `map ((item) -> item.key)` but more concise.



10. PRO TIP

- ✓ Use `pluck()` for simple field extraction.
- ✓ For complex logic, prefer `map()` with expressions.
- ✓ Combine `pluck()` with other functions like `filter()`, `groupBy()`, `orderBy()` for powerful transformations.
- ✓ It improves code readability and reduces lines of code.



Remember: `pluck()` is your shortcut to extract a single field from every Object in an Array. Simple. Clean. Powerful!



BONUS: EQUIVALENT USING `map()`

```
payload map ((item) -> item.name)
// Same as: payload pluck ("name")
```





DATAWEAVE FUNCTION

flatten()



Think Smart!

Use `flatten()` to convert multi-level Arrays into a single-level Array.

Flattens nested Arrays into a single-level Array.

1. WHAT DOES IT DO?

The `flatten()` function is used to convert a nested Array (Array of Arrays) into a single-level Array.



2. WHEN DO WE USE IT?

- When we receive a nested Array and need a flat list.
- When merging multiple lists into a single list.
- When we want to simplify nested structures for further processing.



3. SYNTAX

```
payload flatten()
// or
payload flatten
```

- No parameters.
- Works only on Arrays.

4. IMPORTANT POINTS

- Always returns a new Array.
- Does not modify the original Array.
- Flattens **one** level of nesting at a time.
- Use multiple `flatten()` calls for deeper nesting.



5. EASY EXAMPLE

Input (Payload)

```
[
  [1, 2, 3],
  [4, 5],
  [6]
]
```

DataWeave Code

```
payload flatten()
```

Output

```
[1, 2, 3, 4, 5, 6]
```



6. MEDIUM EXAMPLE

Input (Payload)

```
[
  ["Apple", "Banana"],
  ["Orange", "Grapes"],
  ["Mango"]
]
```

DataWeave Code

```
payload flatten()
```

Output

```
["Apple", "Banana", "Orange", "Grapes", "Mango"]
```



7. DIFFICULT (REAL-TIME SCENARIO)

Requirement:
From an API response, return all tag names in a single list. Tags are grouped under categories.

Input (Payload)

```
[
  { "category": "Fruits", "tags": ["Apple", "Banana"] },
  { "category": "Citrus", "tags": ["Orange", "Lemon"] },
  { "category": "Tropical", "tags": ["Mango", "Pineapple"] }
]
```

DataWeave Code

```
payload map $.tags flatten()
```

Output

```
["Apple", "Banana", "Orange", "Lemon", "Mango", "Pineapple"]
```



8. COMMON MISTAKE

Using `flatten()` on a non-Array or on an already flat Array.

```
payload.name flatten()
// Error: payload.name is not an Array
```

 Tip: Make sure the payload is an Array of Arrays before using `flatten()`.

9. INTERVIEW TIP

Be ready to explain:

- What `flatten()` does.
- How it works with one-level and multi-level nesting.
- How to combine with `map()`, `filter()`, `groupBy()`, etc.

Example Explanation:
`flatten()` removes one level of nesting. For deeper nesting, apply it multiple times.

10. PRO TIP

- Use `flatten()` to simplify data before further transformations.
- Combine with `map()` to extract lists from objects before flattening.
- Great for handling arrays returned from multiple sources.
- Improves readability and reduces complex loops.

 **Remember:** `flatten()` is your tool to turn nested Arrays into a clean, single-level Array. Simple. Fast. Powerful!



BONUS: DEEP FLATTEN

For multi-level nesting, chain `flatten()`.

```
payload flatten() flatten() flatten()
// Flattens multiple levels
```





DATAWEAVE FUNCTION

mapObject()

Think Smart! 

Use `mapObject()` when you want to transform all values of an Object while keeping the same keys.

Transforms each value of an **Object** using an expression and returns a new **Object** with the same keys.

1. WHAT DOES IT DO?

The `mapObject()` function is used to transform all values of an **Object** using a given expression and returns a new **Object** with the same keys.



2. WHEN DO WE USE IT?

- ✓ When we want to modify all values in an **Object**.
- ✓ When we want to perform the same operation on every value.
- ✓ When we want to transform data without changing the keys.



3. SYNTAX

payload `mapObject` ((value, key) -> expression)

- **value** -> Current value of the Object
- **key** -> Current key of the Object
- **expression** -> Transformation logic for each value

4. IMPORTANT POINTS

- Always returns a new **Object**.
- Does not modify the original **Object**.
- Works only on **Objects**.
- Keys remain the same in the output.



5. EASY EXAMPLE

Input (Payload)

```
{
  "john": 25,
  "alice": 30,
  "bob": 28
}
```

DataWeave Code

```
payload mapObject ((value, key) -> value * 2)
```

Output

```
{
  "john": 50,
  "alice": 60,
  "bob": 56
}
```



6. MEDIUM EXAMPLE

Input (Payload)

```
{
  "john": "john.doe@example.com",
  "alice": "alice@sample.com",
  "bob": "bob@test.com"
}
```

DataWeave Code

```
payload mapObject ((value, key) -> upper(value))
```

Output

```
{
  "john": "JOHN.DOE@EXAMPLE.COM",
  "alice": "ALICE@SAMPLE.COM",
  "bob": "BOB@TEST.COM"
}
```



7. REAL-TIME SCENARIO

Requirement:
From an API response, convert all product prices to include 18% GST.

Input (Payload)

```
{
  "laptop": 1000,
  "mobile": 500,
  "tablet": 300
}
```

DataWeave Code

```
payload mapObject ((value, key) -> (value * 1.18) as Number {format: "0.00"})
```

Output

```
{
  "laptop": 1180.00,
  "mobile": 590.00,
  "tablet": 354.00
}
```



8. COMMON MISTAKE

Using `mapObject()` on an Array instead of an Object.

Wrong Example

```
payload mapObject ((value, key) -> value * 2)
// Error! payload is an Array
```



Tip: `mapObject()` works only on Objects, not on Arrays.

9. INTERVIEW TIP

Be ready to explain:

- ✓ What `mapObject()` does
- ✓ Difference between `map()` and `mapObject()`
- ✓ When to use `mapObject()`
- ✓ How keys and values are handled
- ✓ Real-time use cases



10. PRO TIPS

- ✓ Use `mapObject()` for clean and consistent value transformations.
- ✓ Combine with `upper()`, `lower()`, `trim()`, `replace()`, etc.
- ✓ Great for data normalization and bulk updates.
- ✓ Improves readability and reduces manual coding.

Remember: `mapObject()` is your go-to function when you want to transform all values of an **Object** while keeping the **same** keys. Simple. Smart. Powerful!

BONUS: DIFFERENCE BETWEEN `map()` AND `mapObject()`

`map()`

Works on Arrays.

Transforms each item and returns a new Array.

```
payload map ((item, index) -> item * 2)
```

`mapObject()`

Works on Objects.

Transforms each value and returns a new Object with the same keys.

```
payload mapObject ((value, key) -> upper(value))
```

Try It Yourself!

Take any Object payload and try using `mapObject()` with different functions!



DATAWEAVE FUNCTION

reduce()

Think Smart! ☆

Use `reduce()` when you need to calculate totals, counts, sums, or merge data into a single result.

Reduces an Array to a single value by applying an **accumulator** function on each item.

1. WHAT DOES IT DO?

The `reduce()` function iterates over an **Array**, carrying forward an accumulator value, and returns a **single** result after processing all items.

2. WHEN DO WE USE IT?

- ✓ When we need to calculate totals, sums, averages, or counts.
- ✓ When we want to merge or combine items into a single value.
- ✓ When we need to transform complex data into a simple result.

3. SYNTAX

payload `reduce` ((acc, item, index) -> expression, initialValue)

- **acc** -> Accumulator (carries the result)
- **item** -> Current item in the Array
- **index** -> Current index (optional)
- **expression** -> Logic to update the accumulator
- **initialValue** -> Starting value of accumulator

4. IMPORTANT POINTS

- Always returns a single value.
- Does not modify the original **Array**.
- Initial value is the base for the accumulator.
- Works only on **Arrays**.

5. EASY EXAMPLE

Input (Payload)

```
[ 10, 20, 30, 40 ]
```

DataWeave Code

```
payload reduce ((acc, item) -> acc + item, 0)
```

Explanation

Start with 0.
Add each item to accumulator.
 $(0+10) + 20 + 30 + 40 = 100$

Output

```
100
```

6. MEDIUM EXAMPLE

Input (Payload)

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 28 }
]
```

DataWeave Code

```
payload reduce ((acc, item) -> acc + item.age, 0)
```

Explanation

Start with 0.
Add age of each person.
 $0 + 25 + 30 + 28 = 83$

Output

```
83
```

7. COMPLEX EXAMPLE

Input (Payload)

```
[
  { "category": "Fruits", "amount": 120 },
  { "category": "Fruits", "amount": 80 },
  { "category": "Vegetables", "amount": 90 },
  { "category": "Vegetables", "amount": 110 }
]
```

DataWeave Code

```
payload groupBy $.category pluck {
  (key): $. reduce ((acc, item) -> acc + item.amount, 0)
}
```

Explanation

Group by category.
Use `reduce()` to calculate total amount for each category.

Output

```
{
  "Fruits": 200,
  "Vegetables": 200
}
```

8. COMMON MISTAKES

Using `reduce()` on an **Object** instead of an **Array**.

Wrong Example

```
payload.name reduce ((acc, item) -> acc + item, 0)
// Error! payload.name is not an Array
```

Tip: Make sure your payload is an **Array** before using `reduce()`.

9. INTERVIEW TIP

Be ready to explain:

- What `reduce()` does
- How accumulator works
- Role of initial value
- Difference between `reduce()` and `map()`, `filter()`
- Real-time use cases

10. PRO TIPS

- ✓ Use meaningful names for accumulator (e.g., total, sum, count).
- ✓ Use `reduce()` for aggregations and complex calculations.
- ✓ Combine with `map()`, `filter()`, `groupBy()`, etc. for powerful transformations.
- ✓ Keep the logic inside expression simple and readable.

★ **Remember:** `reduce()` is your secret weapon to turn an Array into a single value. Accumulate. Calculate. Conquer! ♥

BONUS: REAL-TIME SCENARIO

Requirement: From an API response, get the total price of all items in the cart.

Input (Payload)

```
[
  { "item": "Book", "price": 100 },
  { "item": "Pen", "price": 20 },
  { "item": "Bag", "price": 80 }
]
```

DataWeave Code

```
payload reduce ((acc, item) -> acc + item.price, 0)
```

Output

```
200
```

Try It Yourself!

Take any **Array** payload and try using `reduce()` with different initial values and expressions!



DATAWEAVE FUNCTION

reduce()

Think Smart! ☆

Use `reduce()` when you need to calculate totals, sums, counts, or merge data into a single result.

Reduces an **Array** to a single value by iterating over each **item** and using an **accumulator**.

1. WHAT DOES IT DO?

The `reduce()` function iterates over an **Array**, processing each **item** one by one. It uses an **accumulator** to keep track of the intermediate result and returns a single final value.

2. WHEN DO WE USE IT?

- When we need to calculate totals, sums, averages, counts, or any aggregate value.
- When we want to merge or combine items into a single result.
- When we need to transform complex data into a simple, meaningful value.

3. SYNTAX

```
payload reduce (item, accumulator) -> expression
```

- item** -> Current item of the Array
- accumulator** -> Accumulator that carries the intermediate result
- expression** -> Logic to update and return the accumulator

 Note: Initial value of the accumulator is the second parameter.

4. IMPORTANT POINTS

- Always returns a single value.
- Does not modify the original **Array**.
- The **accumulator** is updated in every iteration.
- Works only on **Arrays**.

5. EASY EXAMPLE

Input (Payload)

```
[ 10, 20, 30, 40 ]
```

DataWeave Code

```
payload reduce (item, accumulator) -> accumulator + item, 0
```

Explanation

Start with accumulator = 0.
Add each item to accumulator.
0 + 10 + 20 + 30 + 40 = 100

Output

```
100
```

6. MEDIUM EXAMPLE

Input (Payload)

```
[
  { "name": "John", "age": 25 },
  { "name": "Alice", "age": 30 },
  { "name": "Bob", "age": 28 }
]
```

DataWeave Code

```
payload reduce (item, accumulator) -> accumulator + item.age, 0
```

Explanation

Start with accumulator = 0.
Add each item's age to accumulator.
0 + 25 + 30 + 28 = 83

Output

```
83
```

7. COMPLEX EXAMPLE

Input (Payload)

```
[
  { "category": "Fruits", "amount": 120 },
  { "category": "Fruits", "amount": 80 },
  { "category": "Vegetables", "amount": 90 },
  { "category": "Vegetables", "amount": 110 }
]
```

DataWeave Code

```
payload reduce (item, accumulator) -> accumulator ++ { (item.category): (accumulator[item.category] default 0) + item.amount }, {}
```

Explanation

Start with accumulator = {} (empty object).
For each **item**, add amount to the corresponding category.

Output

```
{
  "Fruits": 200,
  "Vegetables": 200
}
```

8. COMMON MISTAKE

Using `reduce()` on an **Object** instead of an **Array**.

Wrong Example

```
payload reduce (item, accumulator) -> accumulator + item, 0
// Error! payload is not an Array
```

 Tip: Always ensure your payload is an **Array** before using `reduce()`.

9. INTERVIEW TIP

Be ready to explain:

- What `reduce()` does
- How **item** and accumulator work together
- Role of initial accumulator
- Difference between `reduce()` and other functions like `map()`, `filter()`, `groupBy()`
- Real-time use cases

10. PRO TIPS

- Use meaningful initial accumulator based on the expected result type.
- Combine `reduce()` with `map()`, `filter()`, `groupBy()` for powerful data transformations.
- Keep the expression clean and efficient.
- Great for calculations and building summary reports.

Remember: `reduce()` is your go-to function to turn an **Array** into a single value. Accumulate. Calculate. Conquer!

BONUS: REAL-TIME SCENARIO

Requirement:

From an API response, calculate the total price of all items in the cart.

Input (Payload)

```
[
  { "item": "Book", "price": 100 },
  { "item": "Pen", "price": 20 },
  { "item": "Bag", "price": 80 }
]
```

DataWeave Code

```
payload reduce (item, accumulator) -> accumulator + item.price, 0
```

Output

```
200
```

 **Try It Yourself!**

Take any **Array** payload and try using `reduce()` with different initial values and expressions!



DATAWEAVE FUNCTION

splitBy()

Splits a **String** into an **Array** of **Strings** using the specified **separator (delimiter)**.

Think Smart! ★

Use `splitBy()` when you want to break a **String** into parts based on a separator and get an **Array** of **Strings**.

1. WHAT DOES IT DO?

The `splitBy()` function is used to split a **String** into an **Array** of **Strings** based on the given separator (delimiter).

It returns each part as a separate element in the resulting **Array**.

2. WHEN DO WE USE IT?

- ✓ When we need to break a **String** into multiple parts.
- ✓ When we want to process each part separately.
- ✓ When we need to convert a delimited **String** (like CSV) into an **Array**.

3. SYNTAX

```
payload splitBy (separator)
```

- **payload** → String to be split
- **separator** → Delimiter (String) used to split the payload
- **Returns** → Array-<String>

4. IMPORTANT POINTS

- Always returns an **Array**.
- Does not include the separator in the result.
- If separator is not found, returns an **Array** with the original **String** as the only element.
- Works only on **Strings**.

5. EASY EXAMPLE

Input (Payload)

```
"apple,banana,cherry"
```

DataWeave Code

```
payload splitBy(",")
```

Explanation

Splits the String using "," as separator.

Output

```
["apple", "banana", "cherry"]
```

6. MEDIUM EXAMPLE

Input (Payload)

```
"2024-05-12"
```

DataWeave Code

```
payload splitBy("-")
```

Explanation

Splits the date using "-" as separator.

Output

```
["2024", "05", "12"]
```

7. COMPLEX EXAMPLE

Input (Payload)

```
"id=101;name=John;role=Admin"
```

DataWeave Code

```
payload splitBy(";")
```

Explanation

Splits the key-value pairs using ";" as separator.

Output

```
["id=101", "name=John", "role=Admin"]
```

8. COMMON MISTAKE

Using `splitBy()` on non-String payload.

Wrong Example

```
payload splitBy(",")
// Error! payload is not a String
```

Tip: Ensure your payload is a String before using `splitBy()`.

9. INTERVIEW TIP

Be ready to explain:

- What `splitBy()` does
- How it works with different separators
- Edge cases (separator not found, empty String, etc.)
- Difference between `splitBy()` and `replace()` or `substring()`

10. PRO TIPS

- ✓ Use `splitBy()` for CSV parsing, log processing, or token extraction.
- ✓ Combine with `map()`, `filter()`, `trim()` to clean and transform data.
- ✓ Handle empty Strings and missing separators in your logic.
- ✓ Keep your separator simple and consistent for best results.

★ **Remember:** `splitBy()` is the perfect tool to break a String into an **Array** of parts. Simple. Fast. Effective! ♥

BONUS: REAL-TIME SCENARIO

Requirement:

From an API response, split a comma separated list of product tags.

Input (Payload)

```
"electronics,mobile,tablet,accessories"
```

DataWeave Code

```
payload splitBy(",")
```

Output

```
["electronics", "mobile", "tablet", "accessories"]
```

Try It Yourself!

Take any delimited String and try using `splitBy()` with different separators!



DATAWEAVE FUNCTION

joinBy()

Joins all elements of an Array into a single String using the specified separator (delimiter).

Think Smart! ☆

Use `joinBy()` when you want to combine all items of an Array into one String with a separator between them.

1. WHAT DOES IT DO?

The `joinBy()` function is used to join all elements of an Array into a single String, separated by the specified separator (delimiter).

It returns a String. ✓

2. WHEN DO WE USE IT?

- ✓ When we want to combine all elements of an Array into a single String.
- ✓ When we need to create comma-separated values (CSV) or delimited output.
- ✓ When we want a readable String from an Array of values. ☆

3. SYNTAX

`payload joinBy (separator)`

- **payload** -> Array whose elements need to be joined
- **separator** -> Delimiter (String) used between elements
- **Returns** -> String

4. IMPORTANT POINTS

- Always returns a String.
- Does not modify the original Array.
- If separator is an empty String (""), elements are joined without any delimiter.
- Works only on Arrays. ✨

☆ **5. EASY EXAMPLE**

Input (Payload)

```
[ "apple", "banana", "cherry" ]
```

DataWeave Code

```
payload joinBy(",")
```

Explanation
Joins all elements using ",", as separator.

Output

```
"apple,banana,cherry"
```

✓

☆ **6. MEDIUM EXAMPLE**

Input (Payload)

```
[ "2024", "05", "12" ]
```

DataWeave Code

```
payload joinBy("-")
```

Explanation
Joins all elements using "-" as separator.

Output

```
"2024-05-12"
```

✓

☆ **7. COMPLEX EXAMPLE**

Input (Payload)

```
[ "id=101", "name=John", "role=Admin" ]
```

DataWeave Code

```
payload joinBy("&")
```

Explanation
Joins all key-value pairs using "&" as separator.

Output

```
"id=101&name=John&role=Admin"
```

✓

✗ **8. COMMON MISTAKE**

Using `joinBy()` on non-Array payload.

Wrong Example

```
payload joinBy(",")
// Error! payload is not an Array ✗
```

💡 **Tip:** Ensure your payload is an Array before using `joinBy()`.

★ **9. INTERVIEW TIP**

Be ready to explain:

- ✓ What `joinBy()` does
- ✓ How it works with different separators
- ✓ Edge cases (empty Array, empty String separator, single element)
- ✓ Difference between `joinBy()`, `splitBy()`, `replace()`, etc.
- ✓ Real-time use cases 📖

🏆 **10. PRO TIPS**

- ✓ Use `joinBy(",")` for CSV generation.
- ✓ Use `joinBy("")` to concatenate without any separator.
- ✓ Combine with `map()`, `filter()`, `pluck()`, etc., for powerful transformations.
- ✓ Handle empty Arrays to avoid generating unwanted delimiters. ✍️

★ **Remember:** `joinBy()` is your go-to function to convert an Array of values into a single, well-formatted String. Simple. Fast. Powerful! ❤️

BONUS: REAL-TIME SCENARIO

🛒 **Requirement:**
From an API response, create a comma-separated list of product names for a report.

Input (Payload)

```
[ "Laptop", "Mobile", "Tablet", "Watch" ]
```

DataWeave Code

```
payload joinBy(",")
```

Output

```
"Laptop,Mobile,Tablet,Watch"
```

✓

💡 **Try It Yourself!**

Take any Array payload and try using `joinBy()` with different separators!



DATAWEAVE FUNCTION

sizeof()

Think Smart! ☆

Use `sizeof()` when you need to know how many items, keys, or characters are in your data.

Returns the number of elements in an **Array**, the number of key-value pairs in an **Object**, or the length of a **String**.

1. WHAT DOES IT DO?

The `sizeof()` function is used to get the size or length of different data types.

- Array → Number of items
- Object → Number of key-value pairs
- String → Number of characters

It returns a **Number**. ✓

2. WHEN DO WE USE IT?

- ✓ When we need to know how many items are in an **Array**.
- ✓ When we need to know how many key-value pairs are in an **Object**.
- ✓ When we need to know the length of a **String**.
- ✓ When we validate data or set limits based on size. ☆

3. SYNTAX

payload `sizeof`

- payload → Array, Object, or String whose size is to be found
- `sizeof()` → Returns a **Number** representing the size

 Note: Works on Array, Object, and String.

4. IMPORTANT POINTS

- Always returns a **Number**.
- Does not modify the original data.
- For **Object**, it counts the number of keys.
- For **String**, it counts the number of characters.
- Works only on **Array**, **Object**, and **String**. ✨

5. EASY EXAMPLE ☆

Input (Payload)

```
{
  "fruits": ["apple", "banana", "cherry"],
  "user": {"name": "John", "age": 30},
  "city": "Bangalore"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  fruitsSize: payload.fruits sizeof,
  userSize: payload.user sizeof,
  cityLength: payload.city sizeof
}
```

Output

```
{
  "fruitsSize": 3,
  "userSize": 2,
  "cityLength": 9
}
```

✓

6. MEDIUM EXAMPLE ☆

Input (Payload)

```
[ "red", "green", "blue", "yellow", "pink" ]
```

DataWeave Code

```
%dw 2.0
output application/json
---
payload sizeof
```

Explanation

Counts the number of items in the Array.

Output

```
5
```

✓

7. COMPLEX EXAMPLE ☆

Input (Payload)

```
{
  "departments": [
    { "id": 1, "name": "Sales" },
    { "id": 2, "name": "IT" },
    { "id": 3, "name": "HR" }
  ],
  "config": { "env": "prod", "version": "1.0.0" },
  "message": "Welcome to DataWeave!"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  departmentsCount: payload.departments sizeof,
  configKeysCount: payload.config sizeof,
  messageLength: payload.message sizeof
}
```

Output

```
{
  "departmentsCount": 3,
  "configKeysCount": 2,
  "messageLength": 23
}
```

✓

8. COMMON MISTAKE

Using `sizeof()` on unsupported data types like Number, Boolean, or Null.

Wrong Example

```
payload = 123
---
payload sizeof
// Error! sizeof() works only on Array, Object, and String. ✗
```

 Tip: Ensure your payload is an Array, Object, or String.

9. INTERVIEW TIP ☆

Be ready to explain:

- What `sizeof()` does
- How it works with **Array**, **Object**, and **String**
- Difference between `sizeof()` and other functions like `map()`, `filter()`, `pluck()`
- Real-time use cases



10. PRO TIPS 🏆

- ✓ Use `sizeof()` for validations (e.g., max items allowed).
- ✓ Combine with `filter()`, `map()`, `groupBy()` for advanced logic.
- ✓ Great for data quality checks and reporting.
- ✓ Keeps expressions simple and efficient.

✍️

Remember: `sizeof()` gives you the power to know the size or length of your data in one go. Simple. Fast. Handy! ❤️

BONUS: REAL-TIME SCENARIO

 **Requirement:**
From an API response, validate if the number of items in the cart is within the allowed limit.

Input (Payload)

```
{
  "cartId": "C100",
  "items": ["Pen", "Book", "Bag", "Bottle"]
}
```

DataWeave Code

```
%dw 2.0
output application/json
payload.items sizeof <= 5
: { "status": "Valid", "itemCount": payload.items sizeof }
: { "status": "Exceeds Limit", "itemCount": payload.items sizeof }
```

Output

```
{
  "status": "Valid",
  "itemCount": 4
}
```

✓

 **Try It Yourself!**

Try `sizeof()` with different Array, Object, and String payloads and see the results! ✍️



DATAWEAVE FUNCTION

orderBy()

Sorts the elements of an **Array** or the key-value pairs of an **Object** based on the given **criteria**.

Think Smart! ☆

Use `orderBy()` when you need to sort data in ascending or descending order using one or more fields or values.

1. WHAT DOES IT DO?

The `orderBy()` function is used to sort an **Array** of items or the entries of an **Object** based on one or more fields or expressions.

It returns the sorted **Array** or **Object**.

2. WHEN DO WE USE IT?

- When we need to sort an **Array** of objects or values.
- When we need to sort an **Object** by its keys or values.
- When we need multi-level sorting (by multiple fields).
- When we need ascending (asc) or descending (desc) order.

3. SYNTAX

```
payload orderBy (criteria)
```

- payload → Array or Object to be sorted
- criteria → Sorting criteria expression(s)
- Returns → Sorted Array or Object

💡 Default order is ascending (asc).
Use "desc" for descending order.

4. IMPORTANT POINTS

- Always returns a new **Array** or **Object**.
- Does not modify the original data.
- For **Array**, sorting is based on the specified field(s) or value.
- For **Object**, sorting is based on keys or values.
- Works only on **Arrays** and **Objects**.

5. EASY EXAMPLE

Input (Payload)

```
[ { "name": "John", "age": 30 },
  { "name": "Alice", "age": 25 },
  { "name": "Bob", "age": 28 } ]
```

DataWeave Code

```
payload orderBy ((item) -> item.age)
```

Explanation

Sorts the Array of objects by "age" in ascending order.

Output

```
[ { "name": "Alice", "age": 25 },
  { "name": "Bob", "age": 28 },
  { "name": "John", "age": 30 } ]
```

6. MEDIUM EXAMPLE

Input (Payload)

```
[ { "id": 1, "name": "Pen", "price": 20 },
  { "id": 2, "name": "Book", "price": 10 },
  { "id": 3, "name": "Bag", "price": 30 } ]
```

DataWeave Code

```
payload orderBy ((item) -> item.price desc)
```

Explanation

Sorts the Array by "price" in descending order.

Output

```
[ { "id": 3, "name": "Bag", "price": 30 },
  { "id": 1, "name": "Pen", "price": 20 },
  { "id": 2, "name": "Book", "price": 10 } ]
```

7. COMPLEX EXAMPLE

Input (Payload)

```
{
  "employees": [
    { "name": "John", "dept": "IT", "salary": 70000 },
    { "name": "Alice", "dept": "HR", "salary": 60000 },
    { "name": "Bob", "dept": "IT", "salary": 80000 },
    { "name": "Eve", "dept": "HR", "salary": 60000 }
  ]
}
```

DataWeave Code

```
payload.employees orderBy ((item) -> item.dept,
                           (item) -> item.salary desc)
```

Explanation

Sorts by department (asc), and within each department by salary in descending order.

Output

```
[ { "name": "Bob", "dept": "IT", "salary": 80000 },
  { "name": "John", "dept": "IT", "salary": 70000 },
  { "name": "Eve", "dept": "HR", "salary": 60000 },
  { "name": "Alice", "dept": "HR", "salary": 60000 } ]
```

8. COMMON MISTAKE

Using `orderBy()` on unsupported data types like Number, Boolean, or Null.

Wrong Example

```
payload: 123
payload orderBy ((item) -> item)
```

// Error! Works only on Array or Object. ❌

💡 Tip: Ensure your payload is an Array or Object before using `orderBy()`.

9. INTERVIEW TIP

Be ready to explain:

- What `orderBy()` does
- How it works with **Array** and **Object**
- How ascending and descending order works
- Multi-level sorting
- Difference between `orderBy()` and other functions like `groupBy()`, `filter()`, `map()`, etc.
- Real-time use cases

10. PRO TIPS

- Use `orderBy()` with multiple criteria for advanced sorting.
- Default is ascending (asc). Use desc for descending.
- Combine with `map()`, `filter()`, `groupBy()` for powerful data transformations.
- Keep the expressions simple and optimized.

★ **Remember:** `orderBy()` helps you sort your data the right way—quickly, easily, and efficiently! ❤️

BONUS: REAL-TIME SCENARIO

Requirement:

From an API response, return the list of products sorted by price in ascending order.

Input (Payload)

```
[ { "name": "Laptop", "price": 50000 },
  { "name": "Mobile", "price": 15000 },
  { "name": "Tablet", "price": 25000 } ]
```

DataWeave Code

```
payload orderBy ((item) -> item.price)
```

Output

```
[ { "name": "Mobile", "price": 15000 },
  { "name": "Tablet", "price": 25000 },
  { "name": "Laptop", "price": 50000 } ]
```

💡 **Try It Yourself!**

Take any **Array** or **Object** payload and try sorting it with `orderBy()` using different fields and directions!



DATAWEAVE FUNCTION

groupBy()

Think Smart! ☆

Use `groupBy()` when you need to group items in an **Array** based on a key (field or expression) and get an **Object** of groups.

Groups the elements of an **Array** into an **Object** based on the given **key** or **criteria**.

1. WHAT DOES IT DO?

The `groupBy()` function is used to group the elements of an **Array** based on a key (field name or expression).

It returns an **Object** where each key is the group value and the value is an **Array** of items belonging to that group.

2. WHEN DO WE USE IT?

- When we need to group data by a specific field or value.
- When we need to organize records into categories.
- When we need to do aggregations or analysis on grouped data.
- When we need multi-level grouping (by multiple fields).

3. SYNTAX

```
payload groupBy (criteria)
```

- `payload` → Array to be grouped
- `criteria` → Key (field name) or expression used for grouping
- Returns** → Object where each key is the group value and the value is an Array of items

💡 If criteria is an expression, it should return a **String** or **Number**.

4. IMPORTANT POINTS

- Always returns an **Object**.
- Does not modify the original **Array**.
- If key is missing or **null**, items are grouped under the key **"null"**.
- For expressions, the result should be a **String** or **Number**.
- Works only on **Arrays**.

5. EASY EXAMPLE

Input (Payload)

```
{
  { "name": "John", "age": 30, "city": "NY" },
  { "name": "Alice", "age": 25, "city": "LA" },
  { "name": "Bob", "age": 28, "city": "NY" },
  { "name": "Eve", "age": 35, "city": "LA" }
}
```

DataWeave Code

```
payload groupBy $.city
```

Explanation
Groups the users by the "city" field.

Output

```
{
  "NY": [
    { "name": "John", "age": 30, "city": "NY" },
    { "name": "Bob", "age": 28, "city": "NY" }
  ],
  "LA": [
    { "name": "Alice", "age": 25, "city": "LA" },
    { "name": "Eve", "age": 35, "city": "LA" }
  ]
}
```

6. MEDIUM EXAMPLE

Input (Payload)

```
{
  { "id": 1, "category": "Fruits", "amount": 120 },
  { "id": 2, "category": "Fruits", "amount": 80 },
  { "id": 3, "category": "Vegetables", "amount": 90 },
  { "id": 4, "category": "Vegetables", "amount": 110 },
  { "id": 5, "category": "Fruits", "amount": 60 }
}
```

DataWeave Code

```
payload groupBy $.category
```

Explanation
Groups the items by "category".

Output

```
{
  "Fruits": [
    { "id": 1, "category": "Fruits", "amount": 120 },
    { "id": 2, "category": "Fruits", "amount": 80 },
    { "id": 5, "category": "Fruits", "amount": 60 }
  ],
  "Vegetables": [
    { "id": 3, "category": "Vegetables", "amount": 90 },
    { "id": 4, "category": "Vegetables", "amount": 110 }
  ]
}
```

7. COMPLEX EXAMPLE

Input (Payload)

```
{
  { "dept": "IT", "role": "Dev", "name": "John" },
  { "dept": "IT", "role": "Admin", "name": "Alice" },
  { "dept": "HR", "role": "Recruiter", "name": "Bob" },
  { "dept": "HR", "role": "Manager", "name": "Eve" },
  { "dept": "IT", "role": "Dev", "name": "Mike" }
}
```

DataWeave Code

```
payload groupBy {(item) => item.dept ++ "_" ++ item.role }
```

Explanation
Groups by multiple fields (dept and role) using an expression.

Output

```
{
  "IT_Dev": [
    { "dept": "IT", "role": "Dev", "name": "John" },
    { "dept": "IT", "role": "Dev", "name": "Mike" }
  ],
  "IT_Admin": [
    { "dept": "IT", "role": "Admin", "name": "Alice" }
  ],
  "HR_Recruiter": [
    { "dept": "HR", "role": "Recruiter", "name": "Bob" }
  ],
  "HR_Manager": [
    { "dept": "HR", "role": "Manager", "name": "Eve" }
  ]
}
```

8. COMMON MISTAKE

Using `groupBy()` on non-Array payload.

Wrong Example

```
payload: { "city": "NY", "age": 30 }
payload groupBy $.city
// Error! payload is not an Array
```

💡 **Tip:** Always ensure your **payload** is an **Array** before using `groupBy()`.

9. INTERVIEW TIP

Be ready to explain:

- What `groupBy()` does
- How it works with **key** or expression
- Return type (Object of Arrays)
- Handling null or missing keys
- Multi-level grouping using expressions
- Real-time use cases

10. PRO TIPS

- Use `groupBy()` before `map()`, `pluck()`, or `reduce()` for grouped operations.
- Combine with `mapObject()` to transform grouped data.
- Handle null or missing keys gracefully.
- Great for reports, dashboards, and data summaries.

Remember: `groupBy()` helps you organize your data into meaningful groups—making analysis easier and transformations more powerful!

BONUS: REAL-TIME SCENARIO

Requirement:
From an order list, group orders by customerId.

Input (Payload)

```
{
  { "orderId": "01", "customerId": "C1", "amount": 120 },
  { "orderId": "02", "customerId": "C2", "amount": 80 },
  { "orderId": "03", "customerId": "C1", "amount": 90 },
  { "orderId": "04", "customerId": "C3", "amount": 150 },
  { "orderId": "05", "customerId": "C2", "amount": 110 }
}
```

DataWeave Code

```
payload groupBy $.customerId
```

Output

```
{
  "C1": [
    { "orderId": "01", "customerId": "C1", "amount": 120 },
    { "orderId": "03", "customerId": "C1", "amount": 90 }
  ],
  "C2": [
    { "orderId": "02", "customerId": "C2", "amount": 80 },
    { "orderId": "05", "customerId": "C2", "amount": 110 }
  ],
  "C3": [
    { "orderId": "04", "customerId": "C3", "amount": 150 }
  ]
}
```

Try It Yourself!

Take any **Array** payload and try grouping it with `groupBy()` using different keys or expressions!



DATAWEAVE FUNCTION

Think Smart! 

Use `distinctBy()` when you want to remove duplicates from an **Array** based on a specific key or expression.

distinctBy()

Returns a new **Array** with only the **distinct** (unique) items based on a **key** (field or expression).

1. WHAT DOES IT DO?

The `distinctBy()` function is used to return only the first occurrence of items in an **Array** based on the value of a specified key (field name) or expression.

It removes duplicates based on that key or expression. The original order of the first occurrences is preserved. 

2. WHEN DO WE USE IT?

- When you want unique items from an **Array** based on a particular field or value.
- When removing duplicates based on a key while keeping the first occurrence.
- When you need **distinct** results for reporting, validation, or further processing. 

3. SYNTAX

```
payload distinctBy (criteria)
```

- `payload` → Array to process
- `criteria` → Key (field name) or expression used to determine uniqueness
- Returns** → Array with unique items based on criteria

 Keeps the first occurrence of each unique key.

4. IMPORTANT POINTS

- Always returns an **Array**.
- Maintains the original order of the first occurrences.
- Does not modify the original **Array**.
- If key is missing or null, items with the same key (including null) are considered duplicates.
- Works only on **Arrays**. 

5. EASY EXAMPLE

Input (Payload)

```
{
  "id": 1, "name": "John", "city": "NY" },
{ "id": 2, "name": "Alice", "city": "LA" },
{ "id": 1, "name": "John", "city": "NY" },
{ "id": 3, "name": "Bob", "city": "LA" },
{ "id": 2, "name": "Alice", "city": "LA" }
}
```

DataWeave Code

```
payload distinctBy $.id
```

Explanation
Returns unique items based on the "id" field. Keeps the first occurrence of each id.

Output

```
{
  "id": 1, "name": "John", "city": "NY" },
{ "id": 2, "name": "Alice", "city": "LA" },
{ "id": 3, "name": "Bob", "city": "LA" }
}
```



6. MEDIUM EXAMPLE

Input (Payload)

```
{
  "name": "John", "dept": "IT", "salary": 70000 },
{ "name": "Alice", "dept": "HR", "salary": 60000 },
{ "name": "Eve", "dept": "IT", "salary": 75000 },
{ "name": "Bob", "dept": "HR", "salary": 60000 },
{ "name": "Mike", "dept": "IT", "salary": 70000 }
}
```

DataWeave Code

```
payload distinctBy $.dept
```

Explanation
Returns unique departments. Keeps the first record of each department.

Output

```
{
  "name": "John", "dept": "IT", "salary": 70000 },
{ "name": "Alice", "dept": "HR", "salary": 60000 }
}
```



7. COMPLEX EXAMPLE

Input (Payload)

```
{
  "orderId": "01", "customerId": "C1", "amount": 120 },
{ "orderId": "02", "customerId": "C2", "amount": 80 },
{ "orderId": "03", "customerId": "C1", "amount": 150 },
{ "orderId": "04", "customerId": "C3", "amount": 200 },
{ "orderId": "05", "customerId": "C2", "amount": 110 }
}
```

DataWeave Code

```
payload distinctBy $.customerId
```

Explanation
Returns one order per customer (first occurrence).

Output

```
{
  "orderId": "01", "customerId": "C1", "amount": 120 },
{ "orderId": "02", "customerId": "C2", "amount": 80 },
{ "orderId": "04", "customerId": "C3", "amount": 200 }
}
```



8. COMMON MISTAKE

Using `distinctBy()` on non-Array payload.

Wrong Example

```
payload: { "id": 1, "name": "John" }
payload distinctBy $.id
// Error! payload is not an Array 
```

Tip: Always ensure your payload is an **Array** before using `distinctBy()`. 

9. INTERVIEW TIP

Be ready to explain:

- What `distinctBy()` does
- How it works with **Array**
- How it determines uniqueness using a key or expression
- Handling null or missing keys
- Difference between `distinctBy()` and `groupBy()`.
- Real-time use cases 

10. PRO TIPS

- Use `distinctBy()` before `groupBy()` or `reduce()` to optimize performance.
- Combine with `map()`, `filter()`, or `pluck()` for powerful transformations.
- Use expressions for advanced uniqueness (e.g., `$.name ++ "-" ++ $.dept`).
- Great for data cleansing, deduplication, and reporting. 

Remember:  `distinctBy()` helps you get unique records in your **Array** based on a key or expression, keeping only the first occurrence. Simple. Efficient. Powerful! 

BONUS: REAL-TIME SCENARIO

Requirement:
From an API response, return one product per productId (remove duplicates).

Input (Payload)

```
{
  "productId": "P1", "name": "Laptop", "price": 50000 },
{ "productId": "P2", "name": "Mobile", "price": 20000 },
{ "productId": "P1", "name": "Laptop", "price": 50000 },
{ "productId": "P3", "name": "Tablet", "price": 15000 }
}
```

DataWeave Code

```
payload distinctBy $.productId
```



Output

```
{
  "productId": "P1", "name": "Laptop", "price": 50000 },
{ "productId": "P2", "name": "Mobile", "price": 20000 },
{ "productId": "P3", "name": "Tablet", "price": 15000 }
}
```



Try It Yourself! 

Take any **Array** payload and try using `distinctBy()` with different keys or expressions to remove duplicates!



DATAWEAVE FUNCTION

contains()

Checks if an **Array**, **String**, or **Object** contains a specific **value**, **key**, or **substring**.

Think Smart!

Use `contains()` to check existence of an item, key, or substring in your data quickly and easily.

1. WHAT DOES IT DO?

The `contains()` function is used to check if a value exists in an **Array**, a substring exists in a **String**, or a key exists in an **Object**.

It returns a **Boolean** (**true** or **false**).

2. WHEN DO WE USE IT?

- When you need to check if an item exists in an **Array**.
- When you need to check if a substring exists in a **String**.
- When you need to check if a key exists in an **Object**.
- When you need simple presence validation in your data.

3. SYNTAX

```
contains(payload, value)
```

- payload** → **Array**, **String**, or **Object** to search in
- value** → **Value**, **substring**, or **key** to check
- Returns** → **Boolean** (**true** or **false**)

Type of 'value' depends on the type of 'payload'.

4. IMPORTANT POINTS

- Always returns a **Boolean** (**true** or **false**).
- For **Array**, it checks for the existence of a value.
- For **String**, it checks for the existence of a substring.
- For **Object**, it checks for the existence of a key.
- Works only on **Array**, **String**, and **Object**.

5. EASY EXAMPLE

Input (Payload)

```
{
  "fruits": ["apple", "banana", "cherry"],
  "user": { "name": "John", "age": 30 },
  "message": "Hello DataWeave!"
}
```

DataWeave Code

```
contains(payload.fruits, "banana")
contains(payload.user, "name")
contains(payload.message, "DataWeave")
```

Explanation

Checks if "banana" exists in the Array.
Checks if key "name" exists in the Object.
Checks if "DataWeave" exists in the String.

Output

```
[ true, true, true ]
```

6. MEDIUM EXAMPLE

Input (Payload)

```
{
  "items": [10, 20, 30, 40],
  "roles": ["Admin", "User", "Guest"],
  "config": { "env": "prod", "debug": false }
}
```

DataWeave Code

```
contains(payload.items, 20)
contains(payload.roles, "User")
contains(payload.config, "env")
```

Explanation

Checks if number 20 exists in the Array.
Checks if "User" exists in the Array of roles.
Checks if key "env" exists in the Object.

Output

```
[ true, true, true ]
```

7. COMPLEX EXAMPLE

Input (Payload)

```
{
  "orders": [
    { "id": 1, "status": "NEW" },
    { "id": 2, "status": "SHIPPED" },
    { "id": 3, "status": "DELIVERED" }
  ],
  "tags": "fast, reliable, secure",
  "meta": { "count": 3, "source": "API" }
}
```

DataWeave Code

```
contains(payload.orders pluck $.status, "SHIPPED")
contains(payload.tags, "reliable")
contains(payload.meta, "source")
```

Explanation

Checks if "SHIPPED" exists in the list of order statuses.
Checks if substring "reliable" exists in the String.
Checks if key "source" exists in the Object.

Output

```
[ true, true, true ]
```

8. COMMON MISTAKE

Using `contains()` on unsupported data types like **Number**, **Boolean**, or **Null**.

Wrong Example

```
payload: 123
contains(payload, 1)
// Error! contains() works only on Array, String, or Object.
```

Tip: Ensure your payload is an **Array**, **String**, or **Object** before using `contains()`.

9. INTERVIEW TIP

Be ready to explain:

- What `contains()` does
- How it works with **Array**, **String**, and **Object**
- What it returns
- Handling key not found or value not present cases
- Real-time use cases

10. PRO TIPS

- Use `contains()` for quick presence checks and validations.
- Combine with `filter()`, `map()`, `pluck()` for advanced filtering.
- Great for data validation, routing decisions, and conditional logic.
- Keep expressions simple for better performance.

Remember: `contains()` helps you check existence quickly and easily. Simple. Fast. Reliable!

BONUS: REAL-TIME SCENARIO

Requirement:

From an API response, check if a product list contains a specific product name.

Input (Payload)

```
{ "products": ["Laptop", "Mobile", "Tablet", "Watch"] }
```

DataWeave Code

```
contains(payload.products, "Mobile")
```

Output

```
true
```

Try It Yourself!

Take any **Array**, **String**, or **Object** payload and try `contains()` with different values, keys, and substrings!



DATAWEAVE FUNCTION

contains()

Checks if an **Array**, **String**, or **Object** contains a specific **value**, **key**, or **substring**.

Think Smart! 

Use `contains()` to check existence of an item, key, or substring in your data quickly and easily.

1. WHAT DOES IT DO?

The `contains()` function is used to check if a value exists in an **Array**, a substring exists in a **String**, or a key exists in an **Object**.

It returns a **Boolean** (**true** or **false**).



2. WHEN DO WE USE IT?

- When you need to check if an item exists in an **Array**.
- When you need to check if a substring exists in a **String**.
- When you need to check if a key exists in an **Object**.
- When you need simple presence validation in your data.



3. SYNTAX

`contains(payload, value)`

- payload** → **Array**, **String**, or **Object** to search in
- value** → **Value**, substring, or **key** to check
- Returns** → **Boolean** (**true** or **false**)

 Type of 'value' depends on the type of 'payload'.

4. IMPORTANT POINTS

- Always returns a **Boolean** (**true** or **false**).
- For **Array**, it checks for the existence of a value.
- For **String**, it checks for the existence of a substring.
- For **Object**, it checks for the existence of a key.
- Works only on **Array**, **String**, and **Object**.



5. EASY EXAMPLE

Input (Payload)

```
{
  "fruits": ["apple", "banana", "cherry"],
  "user": { "name": "John", "age": 30 },
  "message": "Hello DataWeave!"
}
```

DataWeave Code

```
contains(payload.fruits, "banana")
contains(payload.user, "name")
contains(payload.message, "DataWeave")
```

Explanation

Checks if "banana" exists in the Array.
Checks if key "name" exists in the Object.
Checks if "DataWeave" exists in the String.

Output

```
[ true, true, true ]
```



6. MEDIUM EXAMPLE

Input (Payload)

```
{
  "items": [10, 20, 30, 40],
  "roles": ["Admin", "User", "Guest"],
  "config": { "env": "prod", "debug": false }
}
```

DataWeave Code

```
contains(payload.items, 20)
contains(payload.roles, "User")
contains(payload.config, "env")
```

Explanation

Checks if number 20 exists in the Array.
Checks if "User" exists in the Array of roles.
Checks if key "env" exists in the Object.

Output

```
[ true, true, true ]
```



7. COMPLEX EXAMPLE

Input (Payload)

```
{
  "orders": [
    { "id": 1, "status": "NEW" },
    { "id": 2, "status": "SHIPPED" },
    { "id": 3, "status": "DELIVERED" }
  ],
  "tags": "fast, reliable, secure",
  "meta": { "count": 3, "source": "API" }
}
```

DataWeave Code

```
contains(payload.orders pluck $.status, "SHIPPED")
contains(payload.tags, "reliable")
contains(payload.meta, "source")
```

Explanation

Checks if "SHIPPED" exists in the list of order statuses.
Checks if substring "reliable" exists in the String.
Checks if key "source" exists in the Object.

Output

```
[ true, true, true ]
```



8. COMMON MISTAKE

Using `contains()` on unsupported data types like **Number**, **Boolean**, or **Null**.

Wrong Example

```
payload: 123
contains(payload, 1)
// Error! contains() works only on Array, String, or Object.
```



Tip: Ensure your payload is an **Array**, **String**, or **Object** before using `contains()`.



9. INTERVIEW TIP

Be ready to explain:

- What `contains()` does
- How it works with **Array**, **String**, and **Object**
- What it returns
- Handling key not found or value not present cases
- Real-time use cases



10. PRO TIPS

- Use `contains()` for quick presence checks and validations.
- Combine with `filter()`, `map()`, `pluck()` for advanced filtering.
- Great for data validation, routing decisions, and conditional logic.
- Keep expressions simple for better performance.



Remember: `contains()` helps you check existence quickly and easily. Simple. Fast. Reliable!

BONUS: REAL-TIME SCENARIO

Requirement:
From an API response, check if a product list contains a specific product name.

Input (Payload)

```
{ "products": ["Laptop", "Mobile", "Tablet", "Watch"] }
```

DataWeave Code

```
contains(payload.products, "Mobile")
```

Output

```
true
```



Try It Yourself! 

Take any **Array**, **String**, or **Object** payload and try `contains()` with different values, keys, and substrings!



DATAWEAVE FUNCTION

default()

Returns a **default value** if the given value is **null** or **missing** (not present).

Think Smart!

Use `default()` to provide a fallback value when data is null or missing.

1. WHAT DOES IT DO?

The `default()` function is used to return a default value when the given value is **null** or missing (not present).

If the value is present and not **null**, it returns the original value.

2. WHEN DO WE USE IT?

- When a value might be null.
- When a key or field might be missing.
- When you want to provide a fallback or default value.
- When you need to avoid null values in your output.

3. SYNTAX

```
default(value, defaultValue)
```

- `value` → Value to check
- `defaultValue` → Value to return if value is null or missing
- Returns → value if it is present and not null, otherwise `defaultValue`.

Only **null** or **missing** values are replaced, not other falsy values like 0, false, or empty string.

4. IMPORTANT POINTS

- Returns `defaultValue` only when value is **null** or missing.
- Does not modify the original value if it exists.
- Works with any data type.
- Ideal for setting safe fallback values.
- Works only on a single value (not arrays).

5. EASY EXAMPLE

Input (Payload)

```
{
  "name": "John",
  "age": null,
  "city": "NY"
}
```

DataWeave Code

```
{
  name: payload.name default "Unknown",
  age: payload.age default 25,
  city: payload.city default "USA"
}
```

Explanation

`name` → "John" (present, not null)
`age` → null, so default 25 is used
`city` → missing, so default "USA" is used

Output

```
{
  "name": "John",
  "age": 25,
  "city": "NY"
}
```

6. MEDIUM EXAMPLE

Input (Payload)

```
{
  "user": {
    "id": 101,
    "email": null
  },
  "role": null
}
```

DataWeave Code

```
{
  userId: payload.user.id default 0,
  email: payload.user.email default "no-email@example.com",
  role: payload.role default "Guest"
}
```

Explanation

`user.id` → 101 (present)
`user.email` → null, so default email used
`role` → null, so default "Guest" used

Output

```
{
  "userId": 101,
  "email": "no-email@example.com",
  "role": "Guest"
}
```

7. COMPLEX EXAMPLE

Input (Payload)

```
{
  "orderId": null,
  "customer": {
    "name": null,
    "phone": null
  },
  "items": null,
  "total": 0
}
```

DataWeave Code

```
{
  orderId: payload.orderId default "N/A",
  customerName: payload.customer.name default "Anonymous",
  customerPhone: payload.customer.phone default "000-000-0000",
  items: payload.items default [],
  total: payload.total default 0.0
}
```

Explanation

`orderId` → null, so "N/A" used
`customer.name` → null, so "Anonymous" used
`customer.phone` → null, so "000-000-0000" used
`items` → null, so empty array [] used
`total` → 0 (present), so 0.0 is not used

Output

```
{
  "orderId": "N/A",
  "customerName": "Anonymous",
  "customerPhone": "000-000-0000",
  "items": [],
  "total": 0
}
```

8. COMMON MISTAKE

Forgetting that `default()` works only for null or missing, not for other falsy values.

Wrong Example

```
%cd 2.0
output application/json
---
0 default 10 // Returns 0, not 10
false default true // Returns false, not true
"" default "NA" // Returns "", not "NA"
```

Tip: Use `default()` only for **null** or missing values.

9. INTERVIEW TIP

Be ready to explain:

- What `default()` does
- How it handles null and missing
- Difference between null, missing, and other falsy values
- Use cases for safe fallback values
- Real-time use cases

10. PRO TIPS

- Use `default()` to make your data safe and predictable.
- Combine with `map()`, `filter()`, or `pluck()` for clean transformations.
- Useful for setting default config, environment, or user preferences.
- Keep your default values meaningful and consistent.

Remember: `default()` helps you handle null or missing values safely and provide reliable fallback values.

BONUS: REAL-TIME SCENARIO

Requirement:

From an API response, return user details with default values if some fields are null or missing.

Input (Payload)

```
{
  "id": 2001,
  "name": null,
  "email": null,
  "age": null
}
```

DataWeave Code

```
{
  id: payload.id default 0,
  name: payload.name default "Unknown",
  email: payload.email default "no-email@example.com",
  age: payload.age default 18
}
```

Output

```
{
  "id": 2001,
  "name": "Unknown",
  "email": "no-email@example.com",
  "age": 18
}
```

Try It Yourself!

Take any payload and use `default()` to set safe fallback values for null or missing data.



DATAWEAVE FUNCTION

default()

Think Smart! Use default() to provide a fallback value when data is null or missing.

Returns a default value when the actual value is null or the field is missing.

1. WHAT DOES IT DO?

The **default** keyword is used to provide a fallback value whenever:

- The field is null.
- The field is missing.
- The value is unavailable in the payload.

If the value exists and not null, DataWeave returns the original value.

2. SYNTAX

```
default(actualValue, fallbackValue)
```

4. IMPORTANT POINTS

- Works only for null or missing values.
- Does NOT replace valid values.
- Makes APIs cleaner and more reliable.
- Works with Strings, Numbers, Objects, and Arrays.

One of the most frequently used DataWeave keywords.

5. EASY EXAMPLE

Input

```
{
  "name": "John",
  "age": null,
  "city": "New York"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  name: payload.name default "John",
  age: payload.age default 25,
  city: payload.city default "New York"
}
```

Output

```
{
  "name": "John",
  "age": 25,
  "city": "New York"
}
```

8. COMMON MISTAKES

Many developers think:

```
0 default 100 → 100
```

Wrong! Actual Output: 0

Another example:

```
false default true → true
```

Wrong! Actual Output: false

default does NOT replace:

- 0
- false
- Empty Strings
- Empty Arrays
- Existing values

It only works for:

- Null values
- Missing fields

The **default** keyword is used to provide fallback values whenever a field is null or missing, which helps make MuleSoft APIs more reliable and avoids unnecessary null checks.

Be prepared to explain:

- ✓ Null vs Missing values
- ✓ Why default is better than if-else conditions
- ✓ Real-time use cases of default

TIPS

- ✓ Improves code readability.
- ✓ Makes transformations shorter and cleaner.
- ✓ Use meaningful fallback values for better user experience.

REMEMBER

Null Value? Use default → Missing Field? Use default → Valid Value? Return Original Value

BONUS: REAL-TIME SCENARIO

Requirement: From an API response, return user details with default values if some fields are null or missing.

Input (Payload)

```
{
  "id": 2001,
  "name": null,
  "email": null,
  "age": null
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  id: payload.id default 0,
  name: payload.name default "Unknown",
  email: payload.email default "no-email@example.com",
  age: payload.age default 18
}
```

Output

```
{
  "id": 2001,
  "name": "Unknown",
  "email": "no-email@example.com",
  "age": 18
}
```

TRY IT YOURSELF!

Take any payload and use **default()** to set safe fallback values for null or missing data.



DATAWEAVE FUNCTION

update()

Returns a new **Object** or **Array** with the specified **key / index** updated with a **new value**.

Think Smart! ☆

Use `update()` to update existing fields or elements without changing the original data.

1. WHAT DOES IT DO?

The `update()` function is used to return a new Object or Array with the specified key or index updated with a new value.

It does NOT modify the original data.

A new Object or Array is returned with the changes.

2. WHEN DO WE USE IT?

- When you want to update a field in an Object.
- When you want to update an element in an Array.
- When you want to keep the original data unchanged.
- When you want to create a new updated copy.

3. SYNTAX

For Object

```
object update { key: newValue, ... }
```

For Array

```
array update [ index: newValue, ... ]
```

Examples

```
payload update { name: "John", age: 30 }
payload update [ 1: "Updated Value" ]
```

Object → use { key: value }
Array → use [index: value]

4. IMPORTANT POINTS

- Returns a NEW Object or Array.
- Does not modify the original value.
- Only the specified key / index is updated.
- Works with nested Objects and Arrays.
- If the key / index does not exist, it will be added.

5. EASY EXAMPLE (Object)

Input (Payload)

```
{
  "name": "Alice",
  "age": 25,
  "city": "NY"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
payload update { age: 26, city: "New York" }
```

Explanation

Updates age and city in the Object. Original payload remains unchanged.

Output

```
{
  "name": "Alice",
  "age": 26,
  "city": "New York"
}
```

6. MEDIUM EXAMPLE (Array)

Input (Payload)

```
[
  "Apple",
  "Banana",
  "Cherry"
]
```

DataWeave Code

```
%dw 2.0
output application/json
---
payload update [ 1: "Blueberry" ]
```

Explanation

Updates element at index 1. Indexing starts from 0.

Output

```
[
  "Apple",
  "Blueberry",
  "Cherry"
]
```

7. COMPLEX EXAMPLE (Nested Object)

Input (Payload)

```
{
  "user": {
    "id": 101,
    "name": "Bob",
    "address": {
      "city": "NY",
      "zip": "10001"
    }
  },
  "active": true
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
payload update {
  user: (payload.user update { name: "Robert",
    address: (payload.user.address update { zip: "10002" })),
},
active: false
}
```

Explanation

Updates nested fields (name and zip) and top-level field (active).

Output

```
{
  "user": {
    "id": 101,
    "name": "Robert",
    "address": {
      "city": "NY",
      "zip": "10002"
    }
  },
  "active": false
}
```

8. COMMON MISTAKE

Many developers think `update()` modifies the original data. But it does not!

Wrong Example

```
%dw 2.0
var p = { name: "Sam", age: 20 }
---
p update { age: 21 }
---
p.age // Returns 20 (not 21)
```

Tip: Always use the returned value from `update()`. It returns a new copy.

9. INTERVIEW TIP

Be ready to explain:

- What `update()` does
- Difference between `update()` and direct assignment
- How `update()` works with Objects
- How `update()` works with Arrays
- How `update()` works with nested data
- Real-time use cases

10. PRO TIPS

- Use `update()` to avoid mutating original data.
- Great for immutable transformations.
- Works beautifully with nested data.
- Combine with `map()`, `filter()`, `pluck()`, and `reduce()`.
- If key/index does not exist, it will be added.

★ **REMEMBER:**

```
Original Data (Unchanged) → update() → New Data (Updated Copy) → Safe & Immutable Transformation
```

BONUS: REAL-TIME SCENARIO

Requirement:

Update customer details before sending the response.

Input (Payload)

```
{
  "id": 5001,
  "name": "David",
  "status": "Pending",
  "balance": 100.50
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
payload update { status: "Approved", balance: 150.75 }
```

Output

```
{
  "id": 5001,
  "name": "David",
  "status": "Approved",
  "balance": 150.75
}
```

💡 **TRY IT YOURSELF!**

Take any Object or Array payload and try using `update()` to update fields or elements! Practice makes perfect!



DATAWEAVE FUNCTION DATE FUNCTIONS




Date functions are used to get the current date and time, format dates, and convert values between String and Date formats in DataWeave.

Think Smart!

Use date functions to work with dates, times, and timestamps easily and accurately.

1. WHAT DOES IT DO?

DataWeave Date Functions help you to:

- Get the current date and time.
- Convert String to Date.
- Format dates.
- Extract year, month, and day values.
- Perform date transformations.

2. WHEN DO WE USE IT?

Use Date Functions when:

- APIs return dates in different formats.
- You need to convert a String into a Date.
- You want to display dates in a user-friendly format.
- You need the current timestamp.
- You are working with Salesforce, SAP, or external APIs.

3. SYNTAX

Current Date & Time

```
now()
```

Convert String to Date

```
"2026-07-17" as Date
```

Format Date

```
now() as String {format: "dd-MM-yyyy"}
```

Common Date Formats

- yyyy-MM-dd
- dd-MM/yyyy
- MM/dd/yyyy
- dd/MM/yyyy
- yyyy/MM/dd
- HH:mm:ss

4. IMPORTANT POINTS

- Date transformations are very common in MuleSoft projects.
- DataWeave supports Date, Time and DateTime.
- Always validate the input date format.
- Date functions are frequently asked in MuleSoft interviews.
- Used in almost every enterprise integration project.

5. EASY EXAMPLE

DataWeave Code

```
%dw 2.0
output application/json
---
{
  currentDate: now()
}
```

Output

```
{
  "currentDate": "2026-07-17T10:45:30Z"
}
```

6. MEDIUM EXAMPLE

Requirement: Format today's date.

```
%dw 2.0
output application/json
---
{
  formattedDate:
    now() as String {format: "dd-MM-yyyy"}
}
```

Output

```
{
  "formattedDate": "17-07-2026"
}
```

7. DIFFICULT EXAMPLE (Real-Time Scenario)

Requirement: An external API returns the following date.

Input

```
{
  "orderDate": "2026-07-17"
}
```

Task: Convert it into "17 July 2026".

DataWeave Code

```
%dw 2.0
output application/json
---
{
  orderDate:
    (payload.orderDate as Date)
    as String {format: "dd MMM yyyy"}
}
```

Output

```
{
  "orderDate": "17 July 2026"
}
```

8. COMMON MISTAKE

Incorrect:

```
payload.orderDate as Date
```

When the input is:

```
{
  "orderDate": "17/07/2026"
}
```

This will throw an error because DataWeave does not know the input format.

Correct:

```
(payload.orderDate
as Date {format: "dd/MM/yyyy"})
```

Always specify the format when necessary.

9. INTERVIEW TIP

Interview Questions:

- What is the use of `now()` in DataWeave?
- How do you format dates?
- How do you convert String to Date?
- What are the commonly used date formats?
- How do you handle date transformations in MuleSoft?

Sample Answer:

DataWeave provides built-in support for Date, Time, and DateTime transformations. Functions like `now()`, `as Date`, and `as String` are commonly used to format and transform dates received from external systems.

10. PRO TIPS

- Always validate the input date format.
- Use `now()` for audit and logging purposes.
- Date formatting is very common in Salesforce integrations.
- Prefer meaningful and standardized date formats.
- Practice String to Date and Date to String transformations.



REMEMBER:

Need Current Date? Use `now()` → Need Date Formatting? Use `as String {format: "..."} → Need String to Date? Use as Date → Need Date Transformation? Date ⇌ String`



BONUS: MOST COMMON DATE FUNCTIONS

Function	Description
<code>now()</code>	Returns current DateTime (with time zone).
<code>today()</code>	Returns current date (00:00:00).
<code>((... as Date)).year</code>	Extracts year from date.
<code>((... as Date)).month</code>	Extracts month from date (1-12).
<code>((... as Date)).day</code>	Extracts day from date (1-31).
<code>as Date</code>	Converts String to Date.
<code>as DateTime</code>	Converts value to DateTime.
<code>as String {format: "..."} </code>	Formats Date/DateTime to String.

BONUS: REAL-TIME SCENARIO

Requirement: Convert order date received from the API and send in the required format.

Input (Payload)

```
{
  "orderDate": "2026/07/17 15:25:30"
}
```

DataWeave Code

```
output application/json
---
{
  orderDate: (payload.orderDate as DateTime
    {format: "yyyy/MM/dd HH:mm:ss"})
    as String {format: "dd-MM-yyyy HH:mm:ss"}
}
```

Output

```
{
  "orderDate": "17-07-2026 15:25:30"
}
```

TRY IT YOURSELF!

Take any payload with date values and try using date functions to:

- Get current date
- Format the date
- Convert String to Date
- Extract year, month, and day

Practice makes perfect!



DATAWEAVE FUNCTION

DATE FUNCTIONS - PART 2

(ADVANCED)

Learn how to **extract** and **manipulate** dates in **real-world** MuleSoft projects.

Think Smart!

Use date functions to extract, manipulate and calculate dates like a pro!

1. WHAT DOES IT DO?

Date Functions Part 2 help you to:

- ✓ Extract Year, Month, and Day.
- ✓ Extract Hours, Minutes, and Seconds.
- ✓ Work with Time Zones.
- ✓ Perform Date Arithmetic.
- ✓ Calculate future or previous dates.

2. WHEN DO WE USE IT?

Use these functions when:

- ✓ Calculating expiry dates.
- ✓ Adding or subtracting days.
- ✓ Working with time zones.
- ✓ Building reports and dashboards.
- ✓ Scheduling integrations.
- ✓ Processing timestamps from APIs.

3. SYNTAX

Extract Year
(payload.date as Date).year

Extract Month
(payload.date as Date).month

Extract Day
(payload.date as Date).day

Extract Hour
now().hour

Extract Minutes
now().minutes

Extract Seconds
now().seconds

4. IMPORTANT POINTS

- ✓ Works with Date and DateTime.
- ✓ Very useful in enterprise integrations.
- ✓ Frequently used for reporting and validations.
- ✓ Helps in scheduling and date calculations.
- ✓ Works beautifully with APIs and Databases.

5. EASY EXAMPLE

Input

```
{
  "dob": "1998-05-15"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  year : (payload.dob as Date).year,
  month : (payload.dob as Date).month,
  day : (payload.dob as Date).day
}
```

Output

```
{
  "year" : 1998,
  "month" : 5,
  "day" : 15
}
```

6. MEDIUM EXAMPLE

Requirement : Return the current time.

DataWeave Code

```
%dw 2.0
output application/json
---
{
  hour : now().hour,
  minute : now().minutes,
  second : now().seconds
}
```

Output (example)

```
{
  "hour" : 15,
  "minute" : 35,
  "second" : 20
}
```

7. DIFFICULT EXAMPLE (Real-Time Scenario)

Requirement : Calculate the expiry date after 30 days from today.

DataWeave Code

```
%dw 2.0
output application/json
---
{
  today : now(),
  expiryDate : now() + |P30D|
}
```

Output (example)

```
{
  "today" : "2026-07-17T15:30:45Z",
  "expiryDate" : "2026-08-16T15:30:45Z"
}
```

8. COMMON MISTAKE

Incorrect:

```
now() + 30
```

Wrong! Date values cannot be added directly.

Correct:

```
now() + |P30D|
```

Similarly:

```
now() - |P7D|
```

Subtracts 7 days.

9. INTERVIEW TIP

Interview Questions:

- How do you add 30 days in DataWeave?
- How do you extract Year, Month, Day?
- What is Period notation?
- How do you handle date calculations?
- What functions do you use for DateTime transformations?

Sample Answer:

DataWeave supports Date and DateTime manipulation using built-in properties such as year, month, day, hour, minute, and Period notation like P30D for date calculations.

10. PRO TIPS

- ✓ Use Period notation for date arithmetic.
- ✓ Validate time zones while working with APIs.
- ✓ Use DateTime for timestamps.
- ✓ Standardize date formats across integrations.
- ✓ Date calculations are commonly used in Salesforce and banking integrations.

BONUS EXAMPLES

- Add 7 Days : now() + |P7D|
- Add 3 Months : now() + |P3M|
- Add 1 Year : now() + |P1Y|
- Subtract 15 Days : now() - |P15D|
- Subtract 2 Months : now() - |P2M|
- Add 5 Hours : now() + |PT5H|
- Add 30 Minutes : now() + |PT30M|

MOST COMMON PERIOD NOTATIONS

Notation	Meaning	Notation	Meaning
P1D	1 Day	P3M	3 Months
P7D	7 Days	P1Y	1 Year
P30D	30 Days	PT1H	1 Hour
P1M	1 Month	PT30M	30 Minutes



REMEMBER

Need Year?

.year



Need Month?

.month



Need Day?

.day



Need Current Time?

now()



Practice makes perfect!

Need Future Date?

+ |P30D|



Need Previous Date?

- |P7D|



Need Specific Time?

.hour, .minutes, .seconds





DATAWEAVE FUNCTION

isEmpty()

The **isEmpty()** function checks whether a value is empty and returns either **true** or **false**.

Think Smart!

Use **isEmpty()** to check whether a value is empty. It returns **true** or **false**.

1. WHAT DOES IT DO?

The **isEmpty()** function is used to determine whether:

- An Array is empty.
- A String is empty.
- An Object is empty.
- A Null value is empty.

It returns :

true → If the value is empty
false → If the value contains data

2. WHEN DO WE USE IT?

Use **isEmpty()** when :

- Validating API requests.
- Checking empty payloads.
- Performing input validations.
- Avoiding unnecessary processing.
- Working with optional fields.
- Handling null or blank responses.

3. SYNTAX

Empty Array

`isEmpty([])`

Empty String

`isEmpty("")`

Empty Object

`isEmpty({})`

Null Value

`isEmpty(null)`

Non-Empty Value

`isEmpty(payload.name)`

4. IMPORTANT POINTS

- Returns a Boolean value.
- Works with Arrays, Objects, Strings, and Null.
- Very useful in validations.
- Frequently used in MuleSoft projects.
- Helps avoid Null Pointer issues.

5. EASY EXAMPLE

Input

`[]`

DataWeave Code

```
%dw 2.0
output application/json
---
isEmpty(payload)
```

Output

`true`

6. MEDIUM EXAMPLE

Input

```
{
  "name": "",
  "email": "john@gmail.com"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  nameEmpty : isEmpty(payload.name),
  emailEmpty : isEmpty(payload.email)
}
```

Output

```
{
  "nameEmpty": true,
  "emailEmpty": false
}
```

7. DIFFICULT EXAMPLE (Real-Time Scenario)

Requirement : Validate incoming API data before processing.

Input

```
{
  "customerName": "",
  "country": "India",
  "phone": null,
  "orders": []
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
{
  customerNameEmpty : isEmpty(payload.customerName),
  countryEmpty : isEmpty(payload.country),
  phoneEmpty : isEmpty(payload.phone),
  ordersEmpty : isEmpty(payload.orders)
}
```

Output

```
{
  "customerNameEmpty": true,
  "countryEmpty": false,
  "phoneEmpty": true,
  "ordersEmpty": true
}
```

8. COMMON MISTAKE

Incorrect :

`payload.name == ""`

This **ONLY** works for empty strings.

It **DOES NOT** work for :

`null` `[]` `{}`

Correct :

`isEmpty(payload.name)`

Another Incorrect Example :

`sizeOf(payload.orders) == 0`

Instead use :

`isEmpty(payload.orders)`

Much cleaner and more readable.

9. INTERVIEW TIP

Interview Questions :

- What is **isEmpty()** in DataWeave?
- What data types does it support?
- Does **isEmpty(null)** return true or false?
- Where have you used **isEmpty()** in real projects?

Sample Answer :

The **isEmpty()** function is used to validate whether a String, Array, Object, or Null value is empty. It returns true if the value is empty and false otherwise. It is commonly used in API request validations and conditional logic.



10. PRO TIPS

- ✓ Use **isEmpty()** before processing API payloads.
- ✓ Combine it with if-else statements.
- ✓ Works great with **default**.
- ✓ Frequently used in request validations.
- ✓ Improves readability and maintainability.

BONUS EXAMPLES

Empty String <code>isEmpty("")</code>	Empty Object <code>isEmpty({})</code>	Empty Array <code>isEmpty([])</code>	Null Value <code>isEmpty(null)</code>	Non-Empty Array <code>isEmpty([1, 2, 3])</code>
↓	↓	↓	↓	↓
true	true	true	true	false

REMEMBER

Validate String → `isEmpty()`
Validate Array → `isEmpty()`
Validate Object → `isEmpty()`
Validate Null → `isEmpty()`

Returns
true or false



DATAWEAVE FUNCTION

READ & WRITE FILE

DataWeave provides functions to **READ** and **WRITE** files from the local file system.

Think Smart! ★

Use read and write functions to work with files in DataWeave effectively.

1. WHAT DOES IT DO?

Read and Write file functions allow you to:

- Read content from a file.
- Write content into a file.
- Work with Text, JSON, CSV, XML files.
- Store logs or responses.
- Process files in real time.

2. WHEN DO WE USE IT?

Use these functions when:

- Reading static data from files.
- Writing responses into files.
- Storing logs and audit data.
- Generating reports.
- Working with configuration files.
- Interacting with local file system resources.

3. SYNTAX

Read File

```
read(url: 'file::path/filename', encoding: 'UTF-8')
```

Write File

```
write(payload, 'file::path/filename', encoding: 'UTF-8')
```

Common Encodings

- UTF-8
- US-ASCII
- UTF-16

4. IMPORTANT POINTS

- File path must be correct.
- Reads file content as String.
- Write creates or overrides file.
- Ensure proper permissions.
- Used in logs, reports, and file-based integrations.
- Works only with local file system in Mule runtime.

5. EASY EXAMPLE (Read File)

Requirement: Read the content of a text file.

Assume File Content (data.txt)

```
Hello DataWeave!
Welcome to MuleSoft.
```

DataWeave Code

```
%dw 2.0
output application/json
---
read('file:/tmp/data.txt', encoding: 'UTF-8')
```

Output

```
"Hello DataWeave!\nWelcome to MuleSoft."
```

6. MEDIUM EXAMPLE (Write File)

Requirement: Write JSON data into a file.

DataWeave Code

```
%dw 2.0
output application/json
---
write({
  name: "John",
  age: 30,
  city: "New York"
}, 'file:/tmp/output.json', encoding: 'UTF-8')
```

Result

File 'output.json' will be created in /tmp with the given JSON content.

7. DIFFICULT EXAMPLE (Real-Time Scenario)

Requirement: Read an input file, modify data, and write output to a new file.

Assume Input File (input.json)

```
{
  "id": 101,
  "name": "Alice",
  "status": "Active"
}
```

DataWeave Code

```
%dw 2.0
output application/json
---
var data = read('file:/tmp/input.json', encoding: 'UTF-8')
---
data ++ { processed: true, updatedOn: now() },
'file:/tmp/output-updated.json',
encoding: 'UTF-8'
```

Result

File 'output-updated.json' will contain updated data.

8. COMMON MISTAKES

Incorrect File Path

```
read('tmp/data.txt')
```

Correct

```
read('file:/tmp/data.txt')
```

Incorrect Encoding

```
read('file:/tmp/data.txt')
```

Correct

```
read('file:/tmp/data.txt', encoding: 'UTF-8')
```

Writing Object Without Serialisation

```
write({name: 'John'}, 'file:/tmp/out.json')
```

Correct

```
write({name: 'John'}, 'file:/tmp/out.json', encoding: 'UTF-8')
```

⚠ Always use 'file:' prefix and specify encoding.

9. INTERVIEW TIP

Interview Questions:

- How do you read a file in DataWeave?
- How do you write data into a file?
- What encoding do you use and why?
- Where have you used file functions?

Sample Answer:

DataWeave provides read() and write() functions to read data from a file and write data into a file. These are commonly used for logs, reports, and file-based integrations.

10. PRO TIPS

- Use absolute paths for file operations.
- Always specify encoding.
- Handle exceptions for file not found.
- Write data in formatted structure (JSON, CSV).
- Use in combination with other functions for powerful transformations.

BONUS EXAMPLES

Read JSON File

```
read('file:/tmp/data.json', encoding: 'UTF-8')
```

↓

Returns String content

Read CSV File

```
read('file:/tmp/data.csv', encoding: 'UTF-8')
```

↓

Returns CSV content

Write CSV File

```
write(payload, 'file:/tmp/data.csv', encoding: 'UTF-8')
```

↓

Creates or overwrites file

COMMON FILE PATHS IN MULE

- /tmp → Temporary directory (recommended)
- /apps → Application directory
- /conf → Configuration directory
- /logs → Log files directory

★ **REMEMBER**

Need to Read File? → Use read() → Need to Write File? → Use write() → Work with Local Files Easily!



